Ministerul Educației și Cercetării Al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 5:
Greedy Algorithms: Prim and Kruskal

Elaborated:
st. gr. FAF-243 Poiata Calin

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Table of Contents

ALGORITHM ANALYSIS **3**
Objective 3
Tasks 3
Theoretical Notes 3
Introduction 3
Comparison Metric 4
Input Data 4
IMPLEMENTATION **5**
Prim Algorithm 5
Kruskal Algorithm 6
RESULTS **7**
Execution Time Table 7
Standard Deviation Table 8
Performance Ratios 8
Graphical Analysis 9
Comparative Bar Chart 10
CONCLUSION **11**
ANNEX **12**

ALGORITHM ANALYSIS

Objective

Study the greedy algorithm design technique through implementation and empirical analysis of Prim and Kruskal minimum spanning tree algorithms on sparse and dense graphs.

Tasks

1. Study the greedy algorithm design technique;
2. Implement Prim and Kruskal algorithms in Python;
3. Perform empirical analysis of Prim and Kruskal;
4. Increase the number of graph nodes and analyze the impact on runtime;
5. Present obtained data graphically;
6. Make a report.

Theoretical Notes

Greedy algorithms build a solution step by step by always choosing the locally best option according to a selection rule. Decisions are not rolled back. For some problems this leads to optimal global solutions; for others it can be suboptimal.

For minimum spanning tree (MST) in connected weighted undirected graphs, greedy choice is provably optimal due to cut and cycle properties.

- **Prim** grows a single tree from a start node, repeatedly selecting the minimum-weight edge connecting the current tree to an unvisited node.
- **Kruskal** builds a forest and repeatedly adds the globally lightest edge that does not create a cycle.

Standard methodology:

1. Define the analysis objective (runtime comparison of MST algorithms).
2. Select metrics (average execution time and standard deviation).
3. Define input properties (graph density and number of nodes).
4. Implement the algorithms.
5. Generate connected sparse and dense weighted graphs.
6. Execute multiple trials for increasing graph sizes.
7. Analyze and visualize the measurements.

Introduction

Prim and Kruskal solve the same problem (MST) but follow different greedy strategies. Prim is vertex-growth oriented and relies on a priority queue over frontier edges. Kruskal is edge-order oriented and relies on sorting plus Union-Find cycle detection.

In practice, their performance depends on graph density and scale:

- sparse graphs have fewer candidate edges and lower sorting/heap pressure,
- dense graphs increase the number of edge operations,
- larger node counts amplify these effects.

This laboratory examines these practical differences through controlled experiments.

Comparison Metric

The primary metric is **average wall-clock execution time** (seconds), measured with `time.perf_counter()`. Standard deviation is also recorded to evaluate measurement stability.

Derived ratio metrics are additionally used:

- Prim dense/sparse ratio,
- Kruskal dense/sparse ratio,
- Prim/Kruskal ratio for sparse and dense graphs.

Input Data

The benchmark script uses the following node sizes:

- $N \in \{20, 50, 100, 150, 200, 300\}$

For each size, two connected weighted undirected graphs are generated:

1. **Sparse graph**: approximately $2N$ edges (with connectivity guaranteed by an initial random tree);
2. **Dense graph**: each possible undirected edge is added with probability $p = 0.5$;
3. Edge weights are random integers in $[1, 100]$;
4. Trials per size: 3 trials for $N \leq 150$, 2 trials for $N > 150$;
5. Correctness check: Prim MST total weight is validated against Kruskal MST total weight on every trial.

IMPLEMENTATION

Both algorithms are implemented in Python and executed in the same benchmarking harness. Graphs are generated with deterministic seeds (

```
42 + trial
```

) for reproducibility.

Prim Algorithm

Algorithm Description:

Prim starts from an arbitrary node and repeatedly adds the lightest edge crossing from visited nodes to an unvisited node, until all nodes are included.

Typical complexity with a binary heap and adjacency list: $O(E \log V)$.

Pseudocode:

```
Prim(graph):
    choose start node s
    visited <- {s}
    pq <- all edges incident to s (min-heap by weight)
    mst <- {}

    while |mst| < V - 1:
        (w, u, v) <- extract-min(pq)
        if v already visited: continue
        add edge (u, v, w) to mst
        mark v visited
        push all edges from v to unvisited nodes into pq

    return mst
```

Implementation:

```python
def solve(adjacency):
    n = len(adjacency)
    visited = [False] * n
    min_heap = [(0, -1, 0)]
    mst_weight = 0
    mst_edges = []

    while min_heap and len(mst_edges) < n - 1:
        w, parent, u = heapq.heappop(min_heap)
        if visited[u]:
            continue

        visited[u] = True
        mst_weight += w
        if parent != -1:
            mst_edges.append((parent, u, w))
```

```
        for v, weight in adjacency[u]:
            if not visited[v]:
                heapq.heappush(min_heap, (weight, u, v))

    return mst_weight, mst_edges
```

Kruskal Algorithm

Algorithm Description:

Kruskal sorts all edges by weight and adds them in ascending order if they connect different components. Union-Find is used to detect and avoid cycles efficiently.

Typical complexity: $O(E \log E)$ (dominated by sorting).

Pseudocode:

```
Kruskal(graph):
    sort all edges by weight
    make-set(v) for each vertex v
    mst <- {}

    for each edge (u, v, w) in sorted order:
        if find(u) != find(v):
            add edge (u, v, w) to mst
            union(u, v)
        if |mst| = V - 1: break

    return mst
```

Implementation:

```
def solve(edges, n):
    uf = UnionFind(n)
    mst_weight = 0
    mst_edges = []

    for edge in sorted(edges, key=lambda e: e.w):
        if uf.union(edge.u, edge.v):
            mst_weight += edge.w
            mst_edges.append((edge.u, edge.v, edge.w))
            if len(mst_edges) == n - 1:
                break

    return mst_weight, mst_edges
```

RESULTS

The program was executed for all configured node sizes and both graph densities. Execution reflects direct run of

```
program.py
```

in the project virtual environment.

Execution Time Table

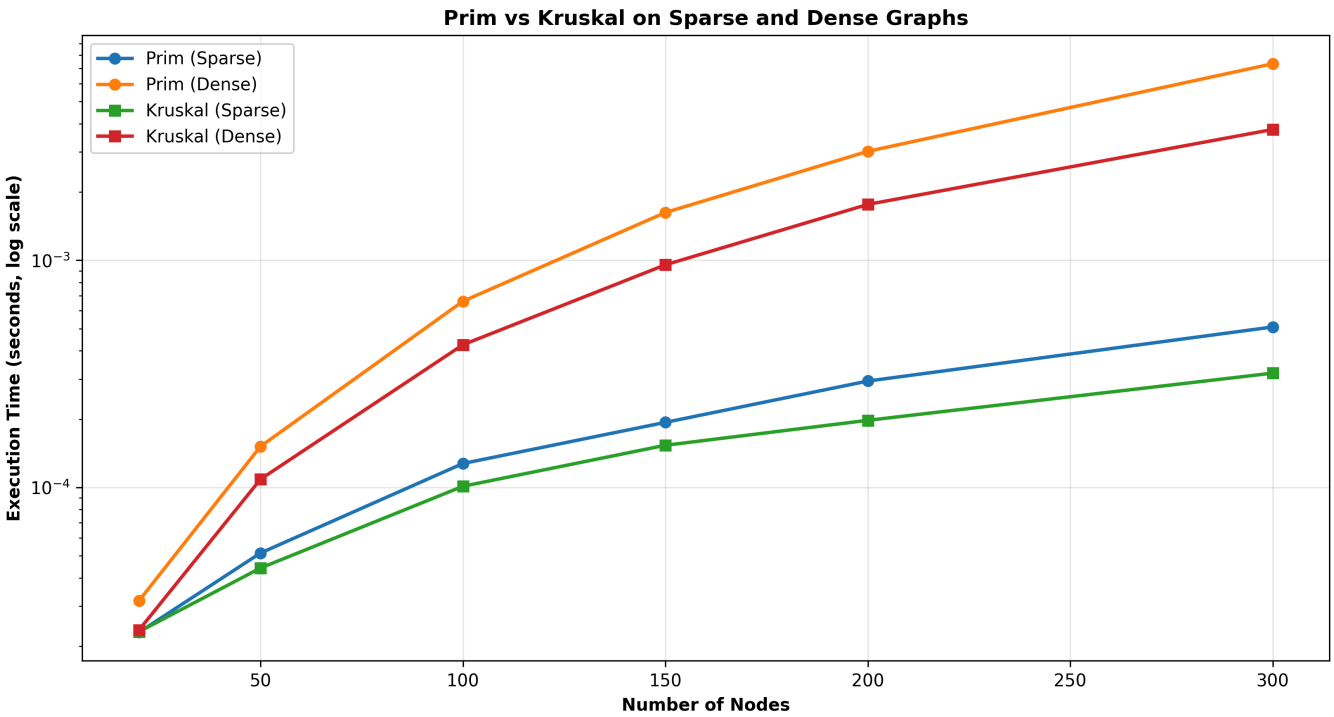

Figure 1 Average execution time (seconds) for Prim and Kruskal

Measured values from the run are:

1. At $N = 300$, Prim: 0.000520 s (sparse), 0.006698 s (dense);
2. At $N = 300$, Kruskal: 0.000333 s (sparse), 0.004161 s (dense);
3. Both algorithms are fast on sparse graphs and significantly slower on dense graphs;
4. Kruskal is generally faster than Prim in the measured configurations.

Standard Deviation Table

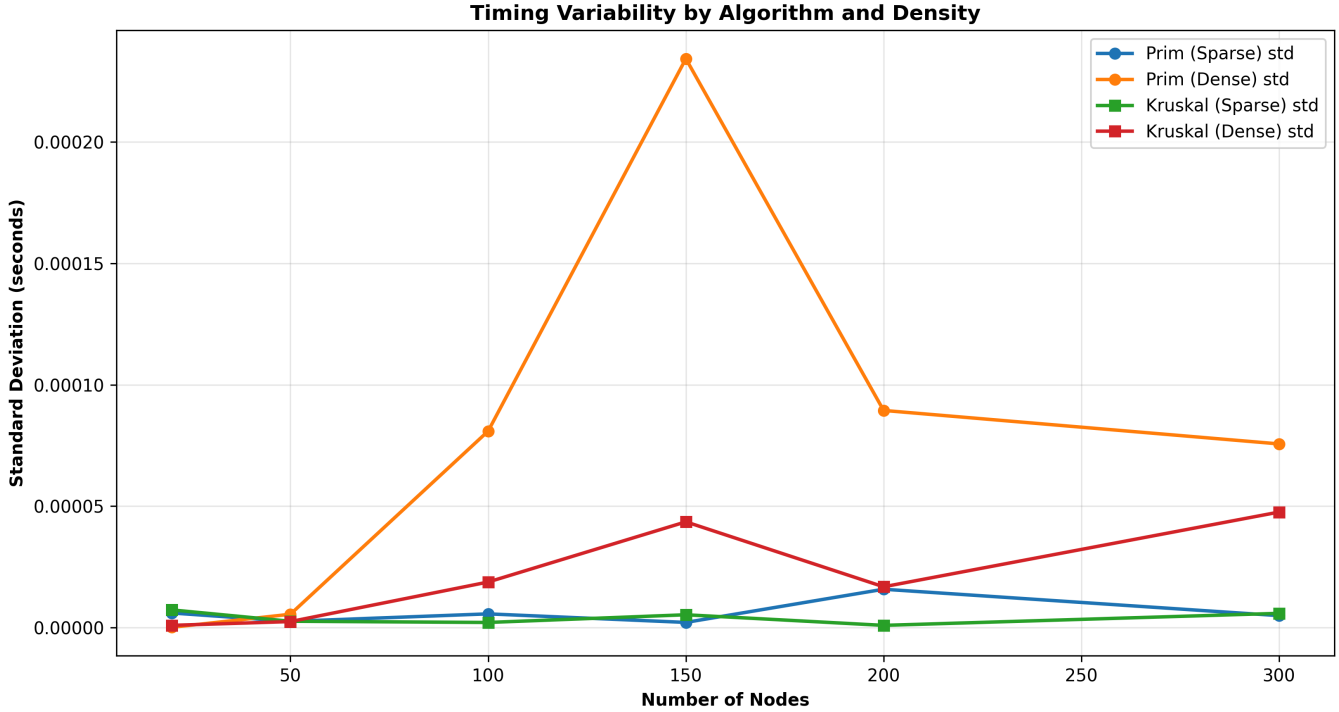

Figure 2 Runtime standard deviations (seconds)

Standard deviations are small relative to the means, indicating stable measurements and consistent trends.

Performance Ratios

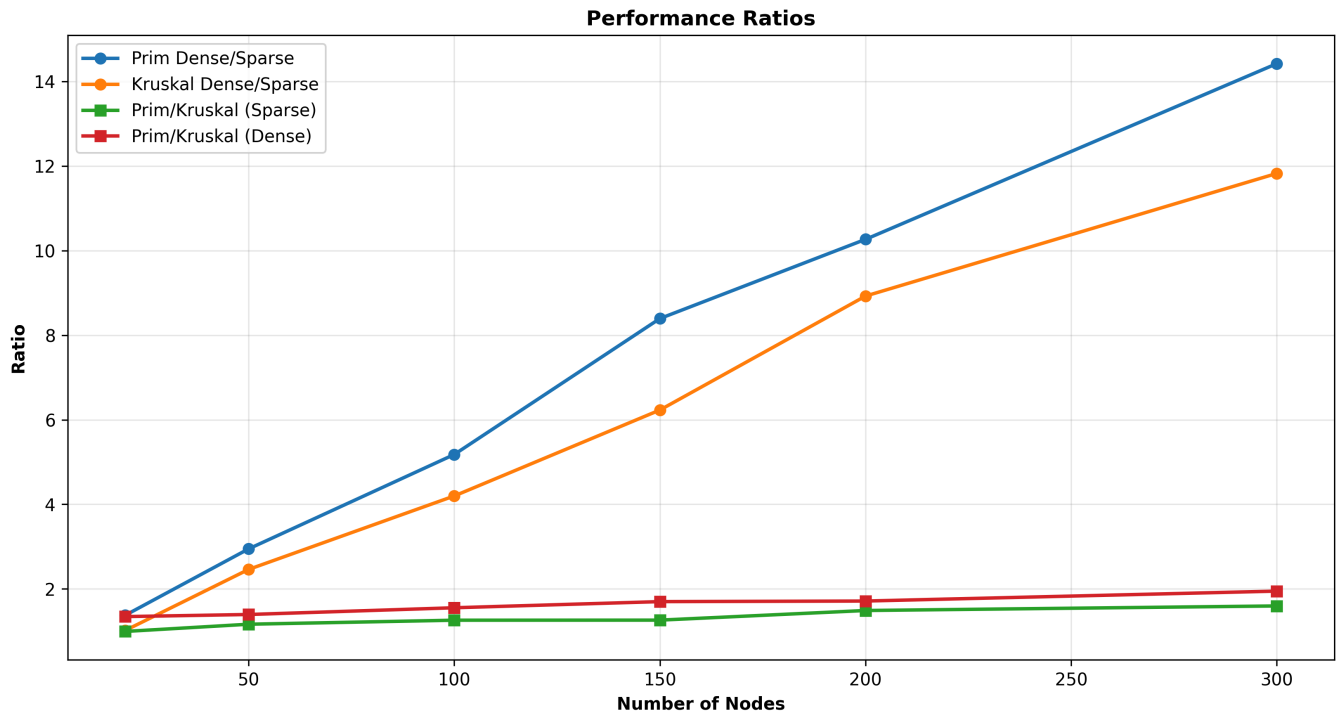

Figure 3 Derived performance ratios (dense/sparse and Prim/Kruskal)

Important ratio-based observations:

1. Prim dense/sparse ratio increases from approximately 1.48x to 12.87x as N grows;
2. Kruskal dense/sparse ratio increases from approximately 1.03x to 12.48x;
3. Prim/Kruskal ratio is above 1 in most cases, showing Kruskal's lower runtime;
4. Density effect becomes much stronger for larger graphs.

Graphical Analysis

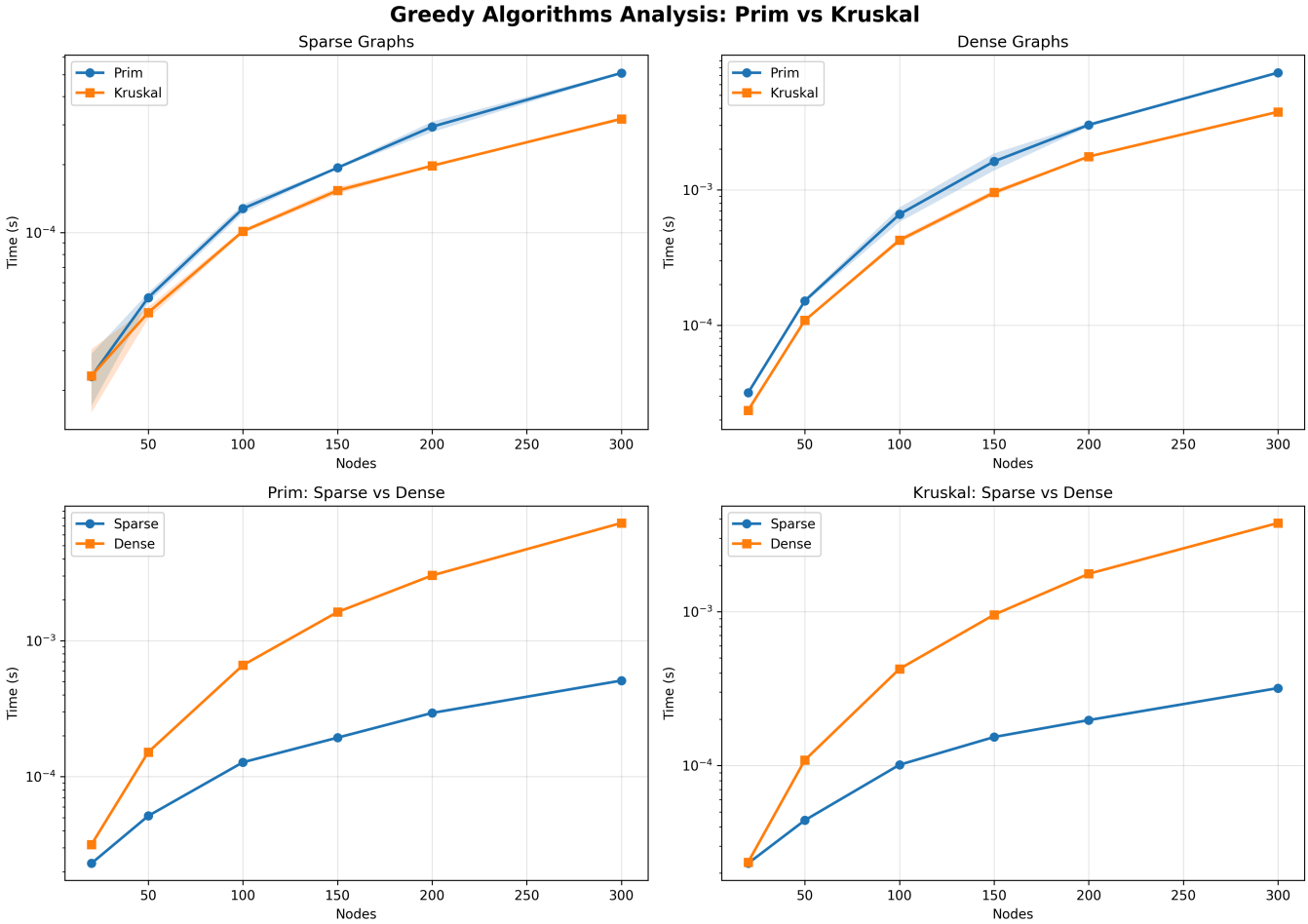

Figure 4 Runtime scaling with node count for sparse and dense graphs

From the plots:

1. Sparse-graph curves grow slowly and remain in sub-millisecond/millisecond range;
2. Dense-graph curves grow much faster for both algorithms;
3. Prim has consistently higher dense-graph runtime than Kruskal in this experiment;
4. Increasing node count amplifies the gap between sparse and dense cases.

Comparative Bar Chart

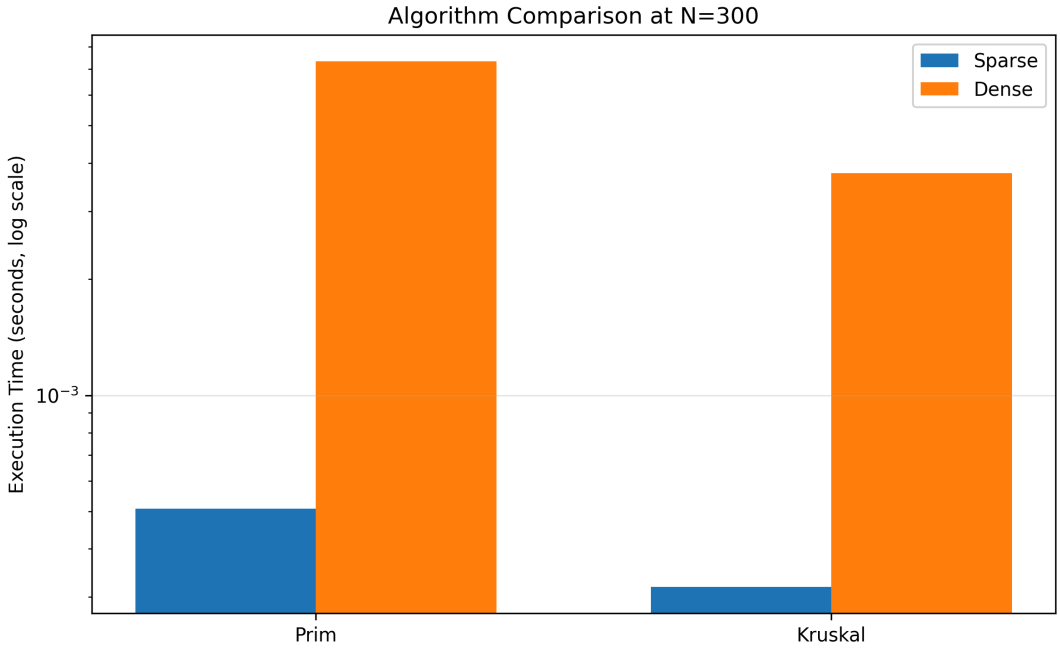

Figure 5 Prim vs Kruskal at the largest tested size

The bar chart at $N = 300$ confirms both algorithms are notably slower on dense graphs and shows Kruskal faster than Prim for both sparse and dense input at that size.

CONCLUSION

The laboratory objectives were achieved. The greedy algorithm design technique was studied, and Prim and Kruskal algorithms were implemented and empirically analyzed on sparse and dense graphs with increasing numbers of nodes. The resulting tables and plots provide a clear practical comparison between the two MST approaches.

The results indicate that both algorithms are strongly influenced by graph density. Sparse graphs lead to very small runtimes, while dense graphs substantially increase execution time due to the larger number of candidate edges processed. This effect becomes more pronounced as the number of nodes increases.

In the performed experiments, Kruskal generally achieved lower runtime than Prim, particularly on larger dense graphs. At the same time, both algorithms produced matching MST total weights on all trials, confirming correctness of the implementations and measurements.

Overall, the empirical findings are consistent with theoretical expectations and show that both algorithms are suitable MST solutions, with practical choice depending on graph structure and performance priorities.

ANNEX

Github Repository